

Tema 1: Máquinas de Turing. Funciones y Lenguajes Calculables (Parte 2)

Serafín Moral

Universidad de Granada

Febrero, 2025

Contenido

- Máquinas de Turing
- Lenguajes recursivamente enumerables. Lenguajes recursivos
- Técnicas de construcción de Máquinas de Turing: memoria adicional, pistas múltiples, subrutinas
- Extensiones del concepto de MT
 - Pasos estáticos
 - Máquinas multicinta
 - Máquinas no deterministas
- Limitaciones de las MT
 - Máquinas con cintas semiilimitadas
- Problemas y Programas
 - Tipos de problemas
 - Programas
 - Problemas de decisión y Lenguajes
 - Palabras y Números
- Calculabilidad
 - El lenguaje diagonal
 - El lenguaje universal
 - Problemas indecidibles
 - Teorema de Rice
 - El problema de las correspondencias de Post

Problemas

Definición

Un problema **PROBLEMA**(x) ó **PROBLEMA** consta de:

- Un conjunto X de **entradas**. Un elemento $x \in X$ se llama una entrada.
- Un conjunto Y de **solución**. Un elemento $y \in Y$ se llama una solución.
- Una aplicación $F : X \rightarrow 2^Y$ que asigna a cada entrada $x \in X$ un conjunto $A \subseteq Y$ de soluciones posibles.

Ejemplo

Búsqueda de caminos en grafos dirigidos:

- Entradas X : conjunto formado por las tripletas (G, ns, nl) , donde G es un grafo dirigido, ns es un nodo de salida, nl es un nodo de llegada.
- Conjunto Y : lista de nodos $(n_1, \dots, n_k)$
- $F(G, ns, nl)$ es el conjunto de las listas de nodos $(n_1, \dots, n_k)$ tales que $n_1 = ns, n_k = nl$ y todas las parejas (n_i, n_{i+1}) sean arcos de G .

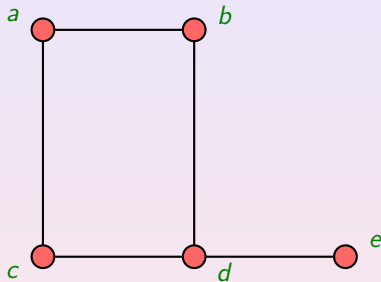
Problemas y su codificación

- Si queremos resolver un problema en un ordenador habrá que codificar las entradas y las soluciones en unos datos que pueda entender el ordenador.
- Nosotros vamos a suponer que las entradas y soluciones están siempre codificadas como palabras o cadenas de caracteres.
- Esto no supone ninguna restricción ya que cualquier tipo de entrada a un programa se puede representar mediante una cadena de caracteres, así como cualquier tipo de salida.
- También supondremos que hay sólo una palabra de entrada y una palabra de salida. Esto tampoco supone restricción ya que un conjunto de palabras se pueden codificar como una sola palabra en la que aparecen las palabras originales encadenadas con un separador para indicar el final de una y el comienzo de la siguiente.
- A partir de ahora, en términos generales hablaremos sólo sobre problemas en las que las entradas y las salidas son palabras sobre un alfabeto (**Problema Computacional**), aunque al hablar de un problema concreto hablaremos de grafos, números u otros elementos.

Problemas y su Codificación

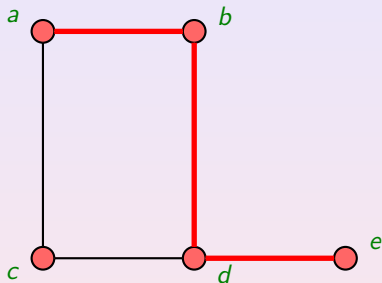
- Cualquier problema se transforma en un problema computacional con un sistema de codificación de las entradas y salidas.
- Puede haber palabras que no sean una codificación correcta de una entrada del problema original. Por ejemplo, si estamos en un problema de grafos, una palabra que no corresponde a una codificación de un grafo. Lo que ocurra con este tipo de entradas es irrelevante para nosotros desde el punto de vista computacional: su detección es un problema siempre simple.
- En general, nuestro estudio teórico se hará sobre problemas computacionales, pero su resultado se aplicará también a problemas en general, ya que estos resultados no dependerán de la codificación elegida (siempre que ésta sea razonable).
- Si tenemos un problema y un sistema de codificación, para cada entrada x , la codificación de x , se denotará como $\langle x \rangle$ (el elemento entre ángulos).

Codificando Grafos



Grafo: *a*,*b* *b*,*d* *c*,*d* *a*,*c* *d*,*e*

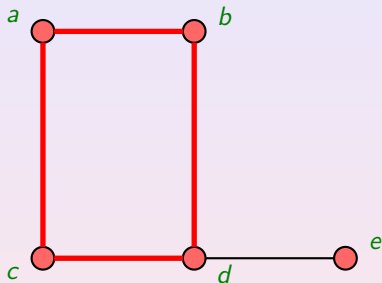
Codificando Grafos



Grafo: a,b b,d c,d a,c d,e

Camino: a,b,d,e

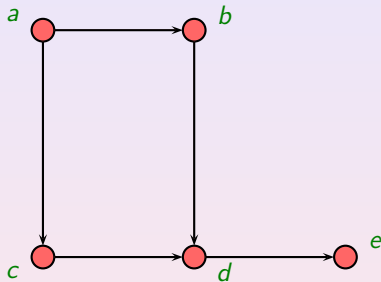
Codificando Grafos



Grafo: *a*,*b* *b*,*d* *c*,*d* *a*,*c* *d*,*e*

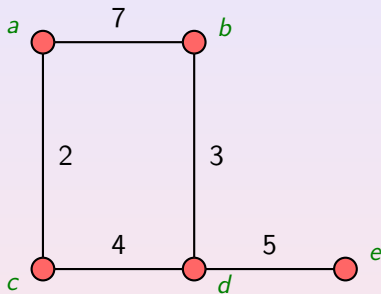
Ciclo: *a*,*b*,*d*,*c*

Codificando Grafos Dirigidos



Grafo dirigido: a,b b,d c,d a,c d,e

Codificando Grafos Con Pesos



Grafo con pesos: a,b,7 b,d,3 c,d,4 a,c,2 d,e,5

Tipos de Problemas

- **Problemas de Búsqueda:** Son los problemas genéricos cuando $F(x)$ puede ser vacío o contener varios elementos: Para una entrada x el problema consiste en encontrar una solución y que cumpla una relación con x cuando este exista y decir 'NO' cuando $F(x) = \emptyset$. Ejemplo: dado un grafo no dirigido encontrar un circuito hamiltoniano.
- **Problemas de Decisión:** Son aquellos en los que las soluciones son $Y = \{SI, NO\}$ y cada entrada x tiene una única solución. Ejemplo: dado un grafo determinar si tiene un circuito hamiltoniano.
- **Problemas de Optimización:** La solución optimiza (minimiza o maximiza) una función definida sobre un conjunto de soluciones factibles asociadas a la entrada. Ejemplo: el problema del viajante de comercio.
- **Problemas de función:** Cada entrada x tiene siempre una y sólo una solución: $F(x)$ tiene un solo elemento. Por ejemplo, dado un número n calcular su cuadrado n^2 . A la solución que corresponde a la entrada x se le denota también por **PROBLEMA(x)** (también en problemas de decisión).

Versión Decisión de los problemas

Los problemas de decisión son especialmente importantes ya que son simples y es fácil razonar sobre ellos y además cualquier otro problema tiene asociado un problema de decisión:

- **Problemas de umbral para problemas de optimización:** Los mismos datos de un problema de optimización más un umbral K y ahora se pregunta si existe una solución de valor mayor o menor que el valor K según sea un problema de máximo o mínimo. Ejemplo: dado un caso del problema del viajante de comercio y un valor K determinar si existe un circuito de coste menor o igual que K .
- **Problemas de existencia para problemas de búsqueda:** Dado un x , determinar si existe un y tal que sea una solución de x .
- **Problemas de comprobación para problemas de función y búsqueda:** Dado x y una posible solución y determinar si y es una solución de x .

- **Camino mínimo:** Dado un grafo y dos nodos, encontrar un camino de longitud mínima entre estos nodos si este existe (problema de optimización).
- **Búsqueda de caminos:** Dado un grafo y dos nodos, encontrar un camino entre ambos nodos, en caso de que exista, decir 'NO' en caso contrario.
- **Existencia de Caminos:** Dado un grafo y dos nodos, determinar si existe un camino entre ellos (problema de existencia).
- **Umbral del camino mínimo:** Dado un grafo, dos nodos, y un umbral K determinar si existe un camino entre estos nodos de longitud menor o igual a K (problema de umbral).
- **Problema de comprobación:** Dado un grafo, dos nodos, y un camino, determinar si es un camino que une esos dos nodos (problema de comprobación).

Un problema computacional de decisión en el que las entradas se codifican como palabras del alfabeto A con el lenguaje de las entradas que tienen respuesta 'SI':

$$L(\text{PROBLEMA}(x)) = \{x \in A^* : \text{PROBLEMA}(x) = 'SI'\}.$$

- Es decir los casos con respuesta **afirmativa**.
- Un lenguaje L sobre un alfabeto A , siempre define también un problema de decisión: dada $x \in A^*$ determinar si $x \in L$.

Un problema de decisión + Una codificación $\equiv$ Problema Computacional 'SI' 'NO'
 $\equiv$ Problema Computacional 'SI' 'NO' $\equiv$ Lenguaje

- Las definiciones sobre lenguajes se pueden transformar en definiciones sobre los problemas asociados. Los problemas cuyos lenguajes son recursivos se llaman **decidibles** o **calculables** y los que no lo son **indecidibles** o **no calculables**. Esta terminología se aplica también a los lenguajes.
- Los problemas cuyos lenguajes son r.e. se llaman **semidecidibles** o **parcialmente calculables**.

Problema Contrario

Dado un problema de decisión **PROBLEMA**, el problema **contrario** de **PROBLEMA**, es el problema **C – PROBLEMA** que intercambia las salidas 'SI' y 'NO' o de forma más precisa

$$\text{PROBLEMA}(x) = 'SI' \Leftrightarrow \text{C – PROBLEMA}(x) = 'NO'.$$

Problemas contrarios y lenguajes

El lenguaje asociado al problema contrario de **PROBLEMA** es el lenguaje complementario del lenguaje del lenguaje asociado a **PROBLEMA**:

$$L(\text{C – PROBLEMA}) = \overline{L(\text{PROBLEMA})}$$

Ejemplo: Isomorfismo de subgrafos

- X : Datos

Dos grafos $G_1 = (V_1, E_1)$ y $G_2 = (V_2, E_2)$

- Y : 'SI', 'NO' (Problema de Decisión)

Relación: Respuesta 'SI' corresponde a ¿Contiene G_1 un subgrafo isomorfo a G_2 ?

Es decir, existe un subconjunto $V' \subseteq V_1$ y un subconjunto de aristas $E' \subseteq E_1$ tal que $E' \subseteq V' \times V'$ y existe una aplicación biyectiva $f : V_2 \rightarrow V'$ de tal manera que se verifica

$$(u, v) \in E_2 \Leftrightarrow (f(u), f(v)) \in E'$$

- Los problemas se resuelven mediante algoritmos.
- ¿Qué es un algoritmo? En esta asignatura vamos a considerar dos conceptos que son equivalentes:
 - Un programa en Python (Ph) bien escrito sintácticamente y con, al menos, una función definida, la primera de las cuales es la función principal (más adecuados para una resolución efectiva de problemas).
 - Una Máquina de Turing (MT) que definiremos en el siguiente tema (más adecuadas para el razonamientos teórico-matemáticos).
- Un algoritmo ALG **resuelve** un problema **PROBLEMA(x)** cuando el argumento de dicho algoritmo es un elemento $x \in X$ y **ALG(x)** es un $y \in F(x)$ o dice “No hay Solución” si $F(x) = \emptyset$.

Codificando Máquinas de Turing

Se le puede asignar a cada MT sobre un alfabeto $\{0,1\}$ una cadena y un número natural. Para ello, suponemos

- Los estados son $\{q_1, \dots, q_k\}$. El estado inicial es q_1 y hay un único estado final q_2 (esto siempre se puede conseguir).
- Los símbolos de B son $\{a_1, a_2, \dots, a_m\}$ donde a_1 es 0, a_2 es 1 y a_3 es el símbolo blanco.
- Al movimiento izquierda le asignamos un 1 y al de la derecha un 2. Este número será $u(M)$.

La codificación de la MT se realiza de la siguiente forma:

- Cada transición $\delta(q_i, a_j) = (q_k, a_l, M)$ se codifica como $0^i 1 0^j 1 0^k 1 0^l 1 0^{u(M)}$.
- Todas las transiciones se van añadiendo a la codificación separadas por 11.

Supondremos que una palabra que no se corresponda a ninguna MT, codifica una MT que siempre rechaza.

Codificando Máquinas de Turing y Entradas

- Para codificar una MT M y una palabra w sobre el alfabeto $\{0,1\}$, podemos codificar M como hemos visto y después añadir 111 seguido de w , dando lugar a la cadena $\langle M, w \rangle$

El lenguaje de diagonalización

Vamos a definir un lenguaje L_d sobre $\{0,1\}$ que no es r.e. Este lenguaje se conoce como **lenguaje de diagonalización**.

Lenguaje de Diagonalización

Sea $w \in \{0,1\}^*$, $w \in L_d$ si y solo si la MT cuya codificación es w , no acepta w .

Si $w_0, w_1, w_2, \dots$ son todas las palabras de $\{0,1\}^*$ ordenadas, entonces

		Palabras				
		w_0	w_1	w_2	w_3	...
Máquinas	w_0	0	0	0	0	...
	w_1	1	0	1	0	...
	w_2	1	1	0	0	...
	w_3	0	1	0	1	...
	.	.	.	.	.	...
	.	.	.	.	.	...
L_d		1	1	1	0	...

$$L_d = \{ \langle M \rangle \mid M \text{ no acepta } \langle M \rangle \text{ como entrada} \}$$

Se supone que si una palabra no es una codificación correcta de una MT, entonces codifica una MT que siempre rechaza (no relevante).

Teorema

L_d no es r.e.

Supongamos una MT M_d que acepta L_d . Dicha MT estará definida sobre el alfabeto $\{0,1\}$. Dicha máquina acepta palabras w tales que la máquina de Turing con codificación w no acepta w .

Sea $\langle M_d \rangle$ la codificación de la MT M_d .

- Si $\langle M_d \rangle \in L_d$ entonces M_d acepta $\langle M_d \rangle$ (ya que M_d acepta L_d), como consecuencia y por la definición de L_d , $\langle M_d \rangle \notin L_d$.
- Si $\langle M_d \rangle \notin L_d$ entonces M_d no acepta $\langle M_d \rangle$ (ya que M_d acepta L_d) y, por la definición de L_d , $\langle M_d \rangle \in L_d$.

Con lo que la existencia de una MT M_d que acepte L_d nos lleva a una contradicción.

Problema de Diagonalización

- A la versión problema de decisión del lenguaje de diagonalización le llamaremos **Problema de Diagonalización** y lo notaremos como **DIAGONAL(M)**.
- Consiste en determinar si dada una máquina de Turing M , entonces M no acepta cuando se lee a ella misma.
- **DIAGONAL(M)** no es semidecidible.

Recordatorio: Problemas y Lenguajes

Dada u , ¿es $u \in L$?

Lenguaje L

Problema Decisión **PROBLEMA**

Conjunto x , es tal que **PROBLEMA**(x) = SI

Máquina Turing Acept.

Aceptar $\equiv$ Responder SI

Algoritmo SI/NO

- Lenguaje recursivamente enumerable $\equiv$ Problema **semidecidible**
 - Existe una MT que acepta las palabras del lenguaje. Para las palabras ^{que no son} del lenguaje la MT puede rechazar o ciclar.
 - Existe un algoritmo que responde correctamente las entradas cuya salida es 'SI', para las entradas de 'NO' el algoritmo puede decir 'NO' o ciclar
- Lenguaje recursivo $\equiv$ Problema **decidible**
 - Existe una MT que acepta las palabras del lenguaje y rechaza las palabras que no son del lenguaje (nunca cicla).
 - Existe un algoritmo que responde correctamente las entradas cuya salida es 'SI' y aquellas cuya salida es 'NO' (nunca cicla).

Complementarios de los lenguajes recursivos y r.e.

Si $L \subseteq A^*$, el complementario de L respecto a A^* se denotará como $\bar{L}$.

Teorema

Si L es un lenguaje recursivo, entonces $\bar{L}$ también lo es. ($\Leftarrow \Rightarrow$)

Demostración

Si L es recursivo, entonces es aceptado por una MT que siempre para. Se construye $\bar{M}$ con las siguientes características:

- Los estados de aceptación de M se convierten en estados de rechazo de $\bar{M}$ donde no se realizan nuevas transiciones.
- $\bar{M}$ tiene un nuevo estado r que es de aceptación y que no tiene transiciones definidas.
- Para cada estado p de M que no sea de aceptación y para cada símbolo de la cinta $a \in B$ para el que no haya definida una transición, se añade $\delta(p, a) = (r, a, D)$.

Está claro que $\bar{M}$ acepta el lenguaje $\bar{L}$ y siempre termina, ya que M lo hace.

Complementarios de los lenguajes recursivos y r.e.

Teorema

Si L y $\bar{L}$ son ambos r.e., entonces L es recursivo. ($\Leftrightarrow$)

Demostración

Sea M_1 la MT que acepta L y M_2 la MT que acepte $\bar{L}$. Vamos a construir una MT M con dos cintas: en una funciona como M_1 y en la otra como M_2 .

M es una máquina que funciona como M_1 y M_2 a la vez (como en el autómata producto). El conjunto de estados de M es el producto $Q_1 \times Q_2$ donde Q_1 es el conjunto de estados de M_1 y Q_2 el de M_2 . En cada paso, se pasa al estado que corresponde según M_1 y al estado que corresponde según M_2 .

La MT termina cuando una de las dos máquinas termina.

Los estados de aceptación de M son las parejas, en las que el primer elemento es un estado de aceptación de M_1 .

Está claro que M acepta L y siempre termina, ya que toda palabra $u \in A^*$ está en L o en $\bar{L}$. En el primer caso, M_1 termina y en el segundo lo hace M_2 . Por lo tanto M siempre termina.

El lenguaje L_d no es r.e. Su complementario podría ser r.e., pero nunca recursivo. De hecho, es r.e.

Los resultados en términos de problemas contrarios

Teorema

Si $\text{PROBLEMA}(x)$ es decidable entonces su contrario $C - \text{PROBLEMA}(x)$ también lo es.

Teorema

Si $\text{PROBLEMA}(x)$ y su contrario $C - \text{PROBLEMA}(x)$ son semidecibles entonces ambos son también decibles.

Resultado

Si $\text{PROBLEMA}(x)$ es semidecible pero no decible entonces su contrario $C - \text{PROBLEMA}(x)$ no es semidecible.

El Lenguaje Universal

Problema Universal

A la versión problema de decisión del lenguaje universal le llamaremos **Problema Universal** y lo notaremos como $\text{UNIVERSAL}(M.w)$.

Consiste en determinar si dada una máquina de Turing M y una entrada w , entonces M acepta w .

Definición

El **lenguaje universal** L_U es el conjunto de todas las palabras v sobre $\{0,1\}$ que son codificaciones de parejas (M, w) (es decir $v = \langle M, w \rangle$) tales que la MT M acepta la cadena w .

Teorema

El problema universal $\text{UNIVERSAL}(M.w)$ es semidecidible.

La idea básica es construir un simulador de MT (una MT universal) M_u que lea la codificación $\langle M, w \rangle$ simule la MT M sobre la entrada w y termine cuando termina M , aceptando si lo hace M . Está claro que si hace la simulación, aceptará cuando M acepta w . M_u contiene varias cintas:

- En la primera contiene la codificación de M y w
- En la segunda contiene lo que sería la cinta de M para la entrada w . Un símbolo $a_i \in B$ se representa como 0^i y los distintos símbolos se separan por un 1 .
- En la tercera cinta representa el estado de M . El estado q_i se representa mediante 0^i
- La cuarta cinta se utiliza para cálculos auxiliares

Demostración (cont.)

- 1 Primero M_u examina M y w para asegurarse de que la entrada es correcta.
- 2 Inicializa la segunda cinta con w . El 0 se codifica como 01. El 1 se codifica como 001. Cada vez que introduzca un blanco lo tendrá que hacer como 0001. Todo de acuerdo con los códigos vistos.
- 3 Inicializa la tercera cinta con 0 que corresponde con el estado inicial (suponemos que es q_1).
- 4 Simula los movimientos. Tendrá que localizar en la primera cinta $0^i 10^j 10^k 10^l 10^m$ donde q_i es el estado en el que se encuentra, $0^i 1$ es lo que se ve en la segunda cinta. Si no lo encuentra para. Entonces debe de ejecutar el movimiento correspondientes:
 - 1 Cambiar el la cinta 3, el estado a 0^k
 - 2 Sustituir 0^j en la cinta 2 por 0^l
 - 3 Hacer el movimiento en la cinta 2, según sea m ($m = 1$ a la izquierda, $m = 2$ a la derecha).
- 5 Si M pasa a un estado de aceptación (el estado q_2), entonces M_u para y acepta.

Demostración en términos de problemas y programas

Teorema

El problema universal $\text{UNIVERSAL}(M, w)$ no es decidable.

Demostración

Por reducción al absurdo:

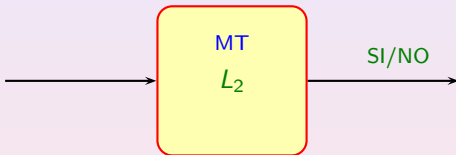
- Si $\text{UNIVERSAL}(M, w)$ fuese decidable, entonces existiría un programa $\text{PROGRAMA}(<M>, w)$ que siempre termina y lo resuelve.
- Construyamos ahora el siguiente programa:

```
PROGRAMAD(<M>)  
  SAL  $\leftarrow$  PROGRAMA(<M>, <M>)  
  Si SAL == 'NO'  
    Return 'SI'  
  En otro Caso  
    Return 'NO'
```

- Pero este programa tiene como entrada una MT M y responde 'SI' cuando esta máquina no acepta su propia codificación, luego implicaría que el problema de diagonalización es decidable, lo que sabemos que no es cierto.

Reducción (en términos de lenguajes)

Si $L_1 \subseteq A^*$ y $L_2 \subseteq B^*$ son lenguajes, el lenguaje L_1 se **reduce** al lenguaje L_2 si existe un algoritmo M (una MT) que siempre para y calcula una función $f : A^* \rightarrow B^*$ tal que para toda entrada $w \in A^*$, $w \in L_1 \Leftrightarrow f(w) \in L_2$.

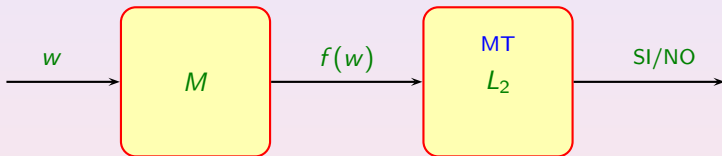


Teorema

Si L_1 se reduce a L_2 , entonces si L_1 no es recursivo, tampoco lo es L_2 y si L_1 no es r.e., tampoco lo es L_2 .

Reducción (en términos de lenguajes)

Si $L_1 \subseteq A^*$ y $L_2 \subseteq B^*$ son lenguajes, el lenguaje L_1 se **reduce** al lenguaje L_2 si existe un algoritmo M (una MT) que siempre para y calcula una función $f : A^* \rightarrow B^*$ tal que para toda entrada $w \in A^*$, $w \in L_1 \Leftrightarrow f(w) \in L_2$.

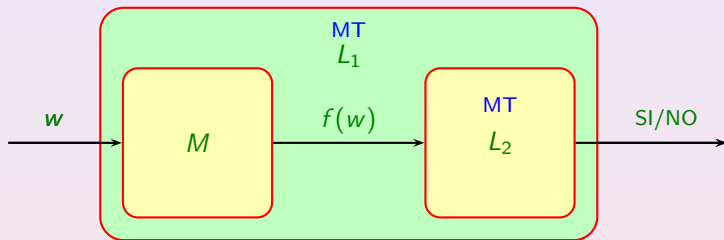


Teorema

Si L_1 se reduce a L_2 , entonces si L_1 no es recursivo, tampoco lo es L_2 y si L_1 no es r.e., tampoco lo es L_2 .

Reducción (en términos de lenguajes)

Si $L_1 \subseteq A^*$ y $L_2 \subseteq B^*$ son lenguajes, el lenguaje L_1 se **reduce** al lenguaje L_2 si existe un algoritmo M (una MT) que siempre para y calcula una función $f: A^* \rightarrow B^*$ tal que para toda entrada $w \in A^*$, $w \in L_1 \Leftrightarrow f(w) \in L_2$. Por tanto, $L_1 \not\subseteq L_2 \Leftrightarrow \overline{L_1} \not\subseteq \overline{L_2}$

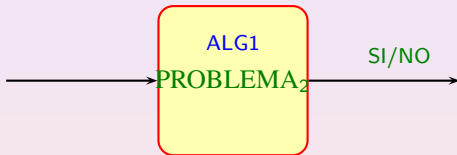


Teorema

Si L_1 se reduce a L_2 , entonces si L_1 no es recursivo, tampoco lo es L_2 y si L_1 no es r.e., tampoco lo es L_2 .

Reducción (en términos de problemas)

Sean **PROBLEMA₁** y **PROBLEMA₂** problemas de decisión, entonces decimos que **PROBLEMA₁** se **reduce** a **PROBLEMA₂** si existe un algoritmo **ALG(*w*)** que siempre para y calcula una función ***f*(*w*)** tal que para toda entrada ***w*** a **PROBLEMA₁**, tenemos que **PROBLEMA₂** produce la misma respuesta para la entrada ***f*(*w*) = ALG(*w*)**.

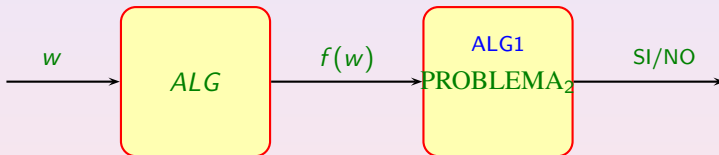


Una **reducción** de **PROBLEMA₁** a **PROBLEMA₂** es un algoritmo **ALG(*x*)** de las entradas de **PROBLEMA₁** a las entradas de **PROBLEMA₂** de tal forma que

$$\text{PROBLEMA}_1(x) = \text{PROBLEMA}_2(\text{ALG}(x))$$

Reducción (en términos de problemas)

Sean **PROBLEMA₁** y **PROBLEMA₂** problemas de decisión, entonces decimos que **PROBLEMA₁** se **reduce** a **PROBLEMA₂** si existe un algoritmo **ALG(w)** que siempre para y calcula una función **f(w)** tal que para toda entrada **w** a **PROBLEMA₁**, tenemos que **PROBLEMA₂** produce la misma respuesta para la entrada **f(w) = ALG(w)**.

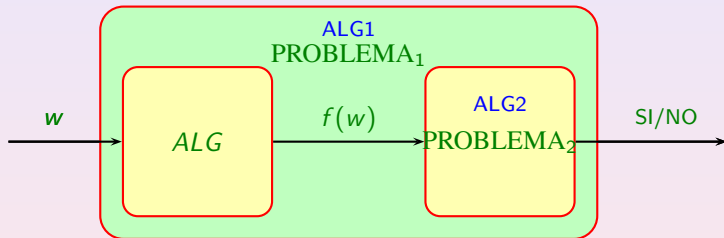


Una **reducción** de **PROBLEMA₁** a **PROBLEMA₂** es un algoritmo **ALG(x)** de las entradas de **PROBLEMA₁** a las entradas de **PROBLEMA₂** de tal forma que

$$\text{PROBLEMA}_1(x) = \text{PROBLEMA}_2(\text{ALG}(x))$$

Reducción (en términos de problemas)

Sean **PROBLEMA₁** y **PROBLEMA₂** problemas de decisión, entonces decimos que **PROBLEMA₁** se **reduce** a **PROBLEMA₂** si existe un algoritmo **ALG(w)** que siempre para y calcula una función **f(w)** tal que para toda entrada **w** a **PROBLEMA₁**, tenemos que **PROBLEMA₂** produce la misma respuesta para la entrada **f(w) = ALG(w)**.



Una **reducción** de **PROBLEMA₁** a **PROBLEMA₂** es un algoritmo **ALG(x)** de las entradas de **PROBLEMA₁** a las entradas de **PROBLEMA₂** de tal forma que

$$\text{PROBLEMA}_1(x) = \text{PROBLEMA}_2(\text{ALG}(x))$$

Reducción (en términos de problemas)

Teorema

Si **PROBLEMA₁** se reduce a **PROBLEMA₂**, entonces si **PROBLEMA₁** es indecidible, también lo es **PROBLEMA₂** y si **PROBLEMA₁** no es semidecidible, tampoco lo es **PROBLEMA₂**.

Sea **ALG(w)** el algoritmo de la reducción de **PROBLEMA₁** a **PROBLEMA₂**. Supongamos que **ALG2(x)** es un algoritmo que hace a **PROBLEMA₂** semidecidible (decidible), entonces el algoritmo:

```
ALG1(x)
  w = ALG(x)
  Return (ALG2(w))
```

hará al problema **PROBLEMA₁** semidecidible (decidible).
Por lo tanto si **PROBLEMA₁** no es semidecidible (decidible),
tampoco lo puede ser **PROBLEMA₂**.

MTs que aceptan el lenguaje vacío

Definimos los siguientes lenguajes sobre el alfabeto $\{0,1\}$:

- L_e conjunto de palabras $\langle M \rangle$ tales que M es una MT sobre $\{0,1\}$ que no acepta ninguna palabra ($L(M) = \emptyset$).
- L_{ne} conjunto de palabras $\langle M \rangle$ tales que M es una MT sobre $\{0,1\}$ que acepta alguna palabra ($L(M) \neq \emptyset$).

Versión Problema

- VACIO(M) es la versión de problema de L_e : dada una MT M , determinar si acepta el lenguaje vacío.
- C-VACIO(M) es la versión de problema de L_{ne} : dada una MT M , determinar si acepta un lenguaje distinto del vacío.

Semidecidibilidad de C-VACIO(M)

Teorema

C-VACIO(M) es semidecidible

Demostración

Basta considerar el siguiente algoritmo no determinista que lo resuelve:

- Eligo de forma no determinista una u palabra sobre $\{0,1\}^*$
- Compruebo con un simulador de MT (máquina de Turing universal) si M acepta u , aceptado en dicho caso y rechazando si no acepta.

Este algoritmo no determinista resuelve C-VACIO(M), ya que si el lenguaje de M no es vacío, al menos una de las opciones no-deterministas acepta. Si es vacío ninguna de la opciones aceptará.

MTs que aceptan el lenguaje vacío

Teorema

C-VACIO(M) no es decidable

Demostración

Vamos a reducir UNIVERSAL(M, w) a C-VACIO(M'):

$$\text{UNIVERSAL}(M, w) \propto \text{C-VACIO}(M')$$

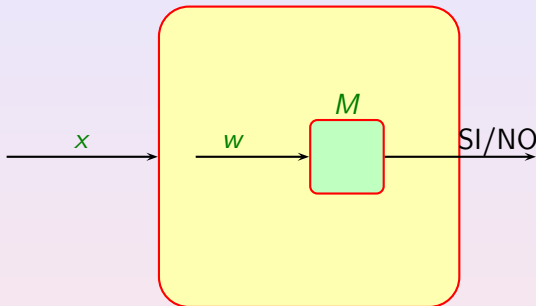
Eso consiste en un algoritmo ALG(M, w) que calcula una MT M' de tal manera que UNIVERSAL(M, w) tenga la misma solución que C-VACIO(M'). Este calcula M' a partir de (M, w) de la siguiente forma:

- M' ignora su entrada x y coloca en la cinta de entrada w . Si la longitud de w es n , esto se puede hacer con n estados. Cada estado q_i escribe el símbolo i de (M, w) y se mueve a la derecha. Después pasaría a un nuevo estado en el que borra lo que quede de x .
- M' se mueve a la izquierda hasta el primer símbolo de w .
- M' pasa al estado inicial de M con w y funciona como M para w .
- La salida de M' es la misma que la de M para w .

Está claro que M acepta w si y solo si M' acepta alguna palabra. De hecho $L(M') = A^*$ si M acepta w y $L(M') = \emptyset$ si M no acepta w .

Demostración Gráfica y Consecuencia

Dada una entrada (M, w) la reducción construye la siguiente MT:



Teorema

$VACIO(M)$ no es semidecidible

Si lo fuese, entonces $C-VACIO(M)$ sería decidable.

Propiedades de los r.e.

Propiedad lenguaje r.e. $\leftrightarrow$ Propiedad de los lenguajes de las MTs

Una propiedad de los lenguajes r.e. se identifica con el problema de saber si el lenguaje de una MT verifica esa propiedad.

Es una problema de decisión del tipo: Dada una MT M , ¿verifica el lenguaje $L(M)$ la propiedad P ?

Ejemplo

Propiedades

Saber si el lenguaje de una MT es finito.

No es una propiedad de los r.e. saber si una MT tiene más de 10 estados.

Definición

Una propiedad de los lenguajes r.e. se dice **trivial** si para toda MT su lenguaje aceptado no verifica la propiedad o para toda MT su lenguaje aceptado siempre verifica la propiedad.

Teorema de Rice

Toda propiedad no trivial sobre los lenguajes r.e. es indecidible

Demostración

Llamemos NOTRIVIAL(M) a dicha propiedad no trivial y supongamos una propiedad no trivial y supongamos que el lenguaje vacío aceptado por la MT M_e no verifica la propiedad que otro lenguaje L aceptado por la MT M_L si verifica la propiedad.

Vamos a reducir el problema UNIVERSAL(M, w) a esta propiedad. Supongamos (M, w) una MT y su entrada construimos M' de la siguiente forma (se supone que tiene una entrada x en la primera cinta):

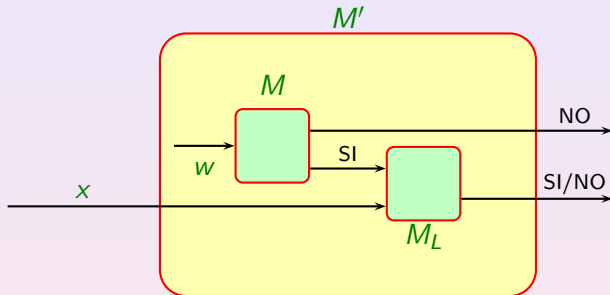
- M' tiene dos cintas. En la segunda coloca a w y empieza trabajando sobre esta cinta con las mismas transiciones de M . Si termina y no acepta, entonces M' no acepta.
- Si M termina y acepta con w , entonces empieza a funcionar como M_L sobre la entrada x en la primera cinta y tiene la misma salida que M_L .

Está claro que si M acepta w , entonces el lenguaje de M' es L y verifica la propiedad. Si M no acepta w , entonces el lenguaje de M' es vacío y no verifica la propiedad. Con esto se acaba la reducción y la propiedad no es decidible.

Si el lenguaje vacío acepta la propiedad se haría la misma transformación con la propiedad complementaria, demostrando que no es decidible y, por lo tanto, la propiedad original tampoco lo es.

Teorema de Rice. Gráfica de la Reducción

Dada una entrada para el problema universal (M, w) se construye la siguiente MT M'



Está claro $\text{UNIVERSAL}(M, w)$ tiene respuesta 'SI', entonces M acepta w , entonces M' acepta L y la respuesta de $\text{NOTRIVIAL}(M')$ es también 'SI'.

Si $\text{UNIVERSAL}(M, w)$ tiene respuesta 'NO', entonces M no acepta w , y esta máquina M' acepta $\emptyset$ y la respuesta de $\text{NOTRIVIAL}(M')$ es también 'NO'.

Como $\text{UNIVERSAL}(M, w)$ no es decidable, $\text{NOTRIVIAL}(M')$ tampoco lo es.

Otros Problemas Indecidibles

El Problema de las Correspondencias de Post (POST) (4.32)

Tenemos un alfabeto de referencia A , y dos listas con la misma longitud $B_1 = w_1, \dots, w_k$, $B_2 = u_1, \dots, u_k$ de palabras sobre A . El problema es determinar si existe una secuencia no vacía de enteros $i_1, \dots, i_m$ tales que $w_{i_1} \dots w_{i_m} = u_{i_1} \dots u_{i_m}$.

Podemos pensar en cada pareja (w_i, u_i) como un bloque de construcción:

w_i
u_i

La especificación del problema nos da un conjunto de bloques disponibles. Por ejemplo:

abb	b	a
a	abb	bb

Otros Problemas Indecidibles

El Problema de las Correspondencias de Post (POST(A1,A2))

Tenemos un alfabeto de referencia A , y dos listas con la misma longitud $B_1 = w_1, \dots, w_k$, $B_2 = u_1, \dots, u_k$ de palabras sobre A . El problema es determinar si existe una secuencia no vacía de enteros $i_1, \dots, i_m$ tales que $w_{i_1} \dots w_{i_m} = u_{i_1} \dots u_{i_m}$.

Podemos pensar en cada pareja (w_i, u_i) como un bloque de construcción:

w_i
u_i

La especificación del problema nos da un conjunto de bloques disponibles. Por ejemplo:

abb	b	a
a	abb	bb

En este caso, la respuesta es afirmativa. Secuencia: 1,3,1,1,3,2,2

abb	a	abb	abb	a	b	b	$\rightarrow$	$abbaabbabbabb$
a	bb	a	a	bb	abb	abb		$abbaabbabbabb$

Si visitáis la página web: <https://people.ksp.sk/~kuko/pcp/> podéis practicar con numerosos ejemplos de este puzzle.

El Problema de las Correspondencias de Post Modificado (POSTM(A1,A2))

PCP Modificado

Tenemos un alfabeto de referencia A , y dos listas con la misma longitud $B_1 = w_1, \dots, w_k$, $B_2 = u_1, \dots, u_k$ de palabras sobre A tales que $u_i, w_j \neq \varepsilon$ y un entero i . El problema es determinar si existe una secuencia no vacía de enteros $i_1, \dots, i_m$ tales que $i_1 = i$ y $w_{i_1} \dots w_{i_m} = u_{i_1} \dots u_{i_m}$.

La única diferencia es que ahora nos dicen el bloque por el que necesariamente hay que comenzar y que las palabras son no vacías. Vamos a suponer siempre que el bloque por el que hay que empezar es el primero de la lista de los bloques (sólo hay que reordenar los bloques para que el que nos indican como bloque por el que hay que empezar aparezca primero).

Ejemplo

Supongamos el ejemplo:

<i>a</i>	<i>ab</i>	<i>bba</i>
<i>baa</i>	<i>aa</i>	<i>bb</i>

Como problema de las correspondencias de Post, tiene solución:
3,2,3,1 (respuesta afirmativa)

Efectivamente sale:

<i>bba</i>	<i>ab</i>	<i>bba</i>	<i>a</i>
<i>bb</i>	<i>aa</i>	<i>bb</i>	<i>baa</i>

Con lo que se lee *bbaabbbbaa* en la parte superior e inferior.
Sin embargo, como PCP modificado no tiene solución (respuesta negativa) ya que en ese caso, una solución tiene que empezar por el primer bloque, y así la palabra superior empieza por *baa* y la inferior por *bb* y ya, pongamos lo que pongamos después, las palabras resultantes no pueden ser iguales.

Semidecidibilidad del probl. de las correspondencias de Post

Se supone que la entrada al problema contiene k bloques



Un algoritmo no-determinista que acepta los casos positivos es:

```
Secuencia = []
Fin = Falso
Mientras No Fin
    Elige  $i$  entre 1 y  $k$ 
    Añade  $i$  a Secuencia
    Elige Fin = Verdadero o Fin = Falso
Comprueba que eligiendo los bloques en el orden Secuencia es una solución del PCP
Si es solución responde 'SI', en caso contrario responde 'NO'
```

El algoritmo resuelve el problema, ya que si la respuesta es afirmativa puede responder de forma afirmativa y si es negativa NUNCA responderá de forma afirmativa.

El Problema de las Correspondencias de Post Modificado

Teorema

El problema de las correspondencias de Post modificado es indecidible si A tiene al menos 2 símbolos.

Si A tiene dos símbolos, podemos codificar palabras de cualquier alfabeto, de manera que sea un homomorfismo y tal que el problema codificado tenga solución si y solo si el original la tiene. Por ejemplo, si el alfabeto es $A = \{a, b, c\}$, podemos codificar $a = 00$, $b = 01$, $c = 11$. Así una ficha se codificaría símbolo a símbolo:

abb	$\rightarrow$	000101
ca		1100

Codificando así todas las fichas, el problema original tiene solución si y solo si el problema codificado con dos símbolos tiene solución.

Vamos a reducir el lenguaje universal $\text{UNIVERSAL}(M, w)$ a este problema.

Vamos a suponer una MT M y una palabra w . La pregunta es si $w \in L(M)$. Vamos a construir un problema de correspondencias de Post modificado con la misma solución.

El alfabeto que vamos a considerar para el alfabeto del problema de las correspondencias es el alfabeto de la MT, más el conjunto de estados, más un separador que no esté en los conjuntos anteriores *

Demostración (Cont.)

- Si $w = a_1 \dots a_n$, introducimos el bloque

$*$
$*q_0 a_1 \dots a_n *$

, que será el bloque de comienzo.
- Por cada transición $\delta(q, a) = (q', b, D)$, introducimos el bloque

qa
bq'
- Por cada transición $\delta(q, a) = (q', b, I)$, introducimos el bloque

cqa
$q'cb$

 para cada c del alfabeto de la MT
- Para cada símbolo a de la MT introducir

a
a
- Los bloques

$*$
$*$

,

$*$
$\#*$

,

$*$
$*\#$
- Para cada $q \in F$ (estado final), añade los bloques

aq
q

,

qa
q

, y el bloque

$q**$
$*$

.

Demostración (Cont.)

- La idea de la reducción es representar en los bloques de una posible solución el cálculo que realiza la MT para la palabra de entrada w escribiendo el historial de sus configuraciones.
- Cada configuración (q, u, v) se representa por la cadena uqv y las distintas configuraciones se separan por $*$.
- En la parte de abajo se lleva una configuración de ventaja respecto a la parte de arriba. Eso ocurre para cada cálculo, pero cuando se llega a un estado de aceptación, entonces la parte de abajo empieza a disminuir símbolo a símbolo paso a paso y copiándose arriba, hasta que quede abajo $\dots * q*$ y arriba $\dots * qa*$ ó $\dots * aq*$ donde q es el estado final. Entonces podemos completar con el último bloque

$q**$
$*$

y se completa la solución.

- Así se obtiene una solución si y solo si el cálculo de la MT llega a un estado de aceptación.
- Como el Problema Universal se reduce al PCP modificado y el problema universal no es decidible, entonces el PCP no es decidible.

Ejemplo

Consideremos la MT

$M = (\{q_0, q_1, q_2, q_3, q_4\}, \{0, 1\}, \{0, 1, X, Y, \#\}, \delta, q_0, \#, \{q_4\})$ donde las transiciones no nulas son las siguientes:

$$\begin{array}{ll} \delta(q_0, 0) = (q_1, X, D) & \delta(q_0, Y) = (q_3, Y, D) \\ \delta(q_1, 0) = (q_1, 0, D) & \delta(q_1, 1) = (q_2, Y, I) \\ \delta(q_1, Y) = (q_1, Y, D) & \delta(q_2, 0) = (q_2, 0, I) \\ \delta(q_2, X) = (q_0, X, D) & \delta(q_2, Y) = (q_2, Y, I) \\ \delta(q_3, Y) = (q_3, Y, D) & \delta(q_3, \#) = (q_4, \#, D) \end{array}$$

y la palabra de entrada 000111

Una solución del PCP modificado, tiene que empezar con la ficha:

*
q ₀ 000111

Ejemplo (Cont.)

*
* q_0 000111*

Cómo $\delta(q_0, 0) = (q_1, X, D)$, tenemos la ficha

$q_0 0$
$X q_1$

 la única forma de proceder en una posible solución es:

*	$q_0 0$	0	0	1	1	1	*
* q_0 000111*	$X q_1$	0	0	1	1	1	*

Si nos fijamos, hemos copiado en la parte superior la configuración que teníamos en la parte inferior y en la inferior se ha añadido la configuración del siguiente paso en el cálculo de la MT. Se puede comprobar que esto es siempre así.

Ejemplo (Cont.)

De esta forma, al final del cálculo de la MT llegaremos a una situación en la que tendremos lo siguiente en las partes superior e inferior de una solución (representamos como una única ficha la solución parcial):

u^*
$u^* XXXYYY \# q_4^*$

donde u es la misma palabra en las dos partes. Podemos ver que aún no es una solución, pero como q_4 es estado final, podemos continuar con los bloques

u^*	X	X	X	Y	Y	Y	$\# q_4$	*
$u^* XXXYYY \# q_4^*$	X	X	X	Y	Y	Y	q_4	*

Si nos damos cuenta, el estado final permite copiar lo que hay de más en la parte inferior en la parte superior, y en la parte inferior se copia todo menos un símbolo que esté al lado del estado final.

Ejemplo (Cont.)

Si vamos repitiendo este proceso, cada vez podemos disminuir lo que hay en la parte inferior en un símbolo de la cinta, hasta que lleguemos a una situación en la que tenemos:

$w*$
$w * q_4*$

Ahora completamos la solución con el bloque

$q_4 * *$
$*$

, que existe por ser q_4 final, obteniendo una solución:

$w*$	$q_4 * *$
$w * q_4*$	$*$

Así se pueden completar dos palabras iguales si y solo si M llega a un estado final cuando tiene a w como entrada.

El PCP es indecidible

Teorema

El problema de las correspondencias de Post es indecidible

Se reduce el problema de las correspondencias de Post modificado al problema de las correspondencias de Post $\text{PCPM}(A1,A2) \propto \text{PCP}(B1,B2)$

Supongamos que nos dan un conjunto de bloques $\begin{bmatrix} u_1 \\ v_1 \end{bmatrix}, \begin{bmatrix} u_2 \\ v_2 \end{bmatrix}, \dots, \begin{bmatrix} u_n \\ v_n \end{bmatrix}$ y que el primero necesariamente es el bloque 1, como un ejemplo del problema $\text{PCPM}(A1,A2)$

Construimos el siguiente problema de Post, $\text{PCP}(B1,B2)$, con dos símbolos nuevos: $\blacklozenge, \blacksquare$ y con los bloques:



Donde, si $u = a_1 a_2 \dots a_n$, se entiende que

$$\blacklozenge u = \blacklozenge a_1 \blacklozenge a_2 \dots \blacklozenge a_n, \quad u \blacklozenge = a_1 \blacklozenge a_2 \dots \blacklozenge a_n, \quad \blacklozenge u \blacklozenge = \blacklozenge a_1 \blacklozenge a_2 \dots \blacklozenge a_n \blacklozenge$$

PCP No decidable. Continuación

- PCPM(B1,B2):
$$\begin{array}{|c|} \hline u_1 \\ \hline v_1 \\ \hline \end{array}, \begin{array}{|c|} \hline u_2 \\ \hline v_2 \\ \hline \end{array}, \dots, \begin{array}{|c|} \hline u_n \\ \hline v_n \\ \hline \end{array}$$

- PCP(B1,B2):

$$\begin{array}{|c|} \hline \blacklozenge u_1 \\ \hline \blacklozenge v_1 \blacklozenge \\ \hline \end{array}, \begin{array}{|c|} \hline \blacklozenge u_1 \\ \hline v_1 \blacklozenge \\ \hline \end{array}, \begin{array}{|c|} \hline \blacklozenge u_2 \\ \hline v_2 \blacklozenge \\ \hline \end{array}, \dots, \begin{array}{|c|} \hline \blacklozenge u_n \\ \hline v_n \blacklozenge \\ \hline \end{array}, \begin{array}{|c|} \hline \blacklozenge \blacksquare \\ \hline \blacksquare \\ \hline \end{array}$$

Como se puede ver, las soluciones a ambos problemas son equivalentes. Si el modificado tiene solución, entonces los bloques asociados con los dos símbolos nuevos son una solución del PCP finalizados por el bloque

$$\begin{array}{|c|} \hline \blacklozenge \blacksquare \\ \hline \blacksquare \\ \hline \end{array}.$$

Por otra parte en cualquier solución del PCP, hay que empezar por la

ficha $\begin{array}{|c|} \hline \blacklozenge u_1 \\ \hline \blacklozenge v_1 \blacklozenge \\ \hline \end{array}$ ya que es la única que comienza en ambas partes por el

mismo símbolo $\blacklozenge$ y además terminar por $\begin{array}{|c|} \hline \blacklozenge \blacksquare \\ \hline \blacksquare \\ \hline \end{array}$. Si a todos los bloques distintos del último les quitamos los dos símbolos especiales obtenemos una solución del PCP modificado que empieza por la ficha 1,

Problemas sobre Gramáticas

Suponemos que G , G_1 y G_2 son gramáticas independientes del contexto dadas y R es un lenguaje regular.

- Saber si $L(G_1) \cap L(G_2) = \emptyset$.
- Determinar si $L(G) = T^*$, donde T es el conjunto de símbolos terminales.
- Comprobar si $L(G_1) = L(G_2)$.
- Determinar si $L(G_1) \subseteq L(G_2)$.
- Determinar si $L(G_1) = R$.
- Comprobar si $L(G)$ es regular.
- Determinar si G es ambigua.
- Conocer si $L(G)$ es inherentemente ambiguo.
- Comprobar si $L(G)$ es determinista.

Teorema

Saber si una gramática independiente del contexto es ambigua (AMBIGUA(G)) es indecidible.

Se reduce el problema de las correspondencias de Post. Supongamos el

alfabeto A y los bloques $\begin{matrix} u_1 \\ v_1 \end{matrix}, \dots, \begin{matrix} u_k \\ v_k \end{matrix}$.

Sea $B = A \cup \{b_1, \dots, b_k\}$ donde $b_i \notin A$ y construimos la siguiente gramática:

$$S \rightarrow C|D$$

$$C \rightarrow u_i C b_i | u_i b_i, \quad i = 1, \dots, k$$

$$D \rightarrow v_i D b_i | v_i b_i, \quad i = 1, \dots, k$$

La solución al problema de las correspondencias de Post es equivalente a que la gramática sea ambigua.

Demostración

Supongamos el alfabeto A y los bloques $\begin{smallmatrix} u_1 \\ v_1 \end{smallmatrix}, \dots, \begin{smallmatrix} u_k \\ v_k \end{smallmatrix}$.

Sea $B = A \cup \{b_1, \dots, b_k\}$ donde $b_i \notin A$ y construimos la siguiente gramática:

$S \rightarrow C|D$, $C \rightarrow u_i C b_i | u_i b_i$, $i = 1, \dots, k$, $D \rightarrow v_i D b_i | v_i b_i$, $i = 1, \dots, k$,

Se basa en lo siguiente:

- Las palabras generadas por la gramática son las generadas a partir de C más las generadas a partir de D
- Las palabras generadas a partir de C son de la forma $u_{i_1} \dots u_{i_l} b_{i_l} \dots b_{i_1}$, donde $i_j \in \{1, \dots, k\}$. Sólo hay una forma de generar una de estas palabras a partir de C .
- Las palabras generadas a partir de D son de la forma $v_{i_1} \dots v_{i_l} b_{i_l} \dots b_{i_1}$, donde $i_j \in \{1, \dots, k\}$. Sólo hay una forma de generar una de estas palabras a partir de D .
- La gramática es ambigua cuando una misma palabra u es generada a partir de C y a partir de D , es decir cuando $u = u_{i_1} \dots u_{i_n} b_{i_n} \dots b_{i_1} = v_{i'_1} \dots v_{i'_{n'}} b_{i'_1} \dots b_{i'_{n'}}$. Esto solo ocurre si $n' = n$ e $i_j = i'_j, \forall j$. Es decir cuando existen $(i_1, \dots, i_n)$ tal que $u_{i_1} \dots u_{i_n} = v_{i_1} \dots v_{i_n}$, es decir cuando el PCP tiene solución.